

Universidad internacional del Ecuador

Facultad de ciencias de la computación

Carrera de ingeniería en software

Evaluación en contacto con el Docente

Selección de programa a desarrollar, generación de diagramas funcionales y arquitectura de software

Estudiante: Ronald Xavier Aguilar Alban

Docente: Mónica Patricia Salazar Tapia

Materia: Lógica de Programación

Paralelo: 1c

Quito– Ecuador

2026

1. Selección del Programa

Programa seleccionado: Generador seguro de contraseñas

2. Análisis del Problema

Problema identificado: Actualmente, la seguridad informática es un aspecto fundamental en el uso de plataformas virtuales, redes sociales, correos e incluso sistemas bancarios. Sin embargo, muchas personas en la actualidad siguen usando contraseñas débiles, repetidas o fáciles de adivina; lo que incrementa el riesgo de accesos no deseados y el robo de información personal.

Uno de los principales problemas identificados es que los usuarios suelen crear contraseñas simples para recordarlas con facilidad, utilizando nombres, fechas de nacimiento, secuencias numéricas o palabras comunes. Estas contraseñas pueden ser vulneradas con extrema facilidad mediante ataques informáticos.

Debido a esta problemática, surge la necesidad de desarrollar un sistema capaz de generar contraseñas seguras de forma automática., Por ello, este programa tiene como objetivo crear claves aleatorias con diferentes niveles de seguridad según las necesidades de este.

El sistema ofrecerá opciones de personalización, permitiendo seleccionar la longitud de la contraseña y definir los tipos de caracteres que el usuario desee.

De esta manera el usuario podrá generar contraseñas robustas y difíciles de descifrar, aumentando la seguridad al momento de encontrarse en la red.

3. Diseño funcional

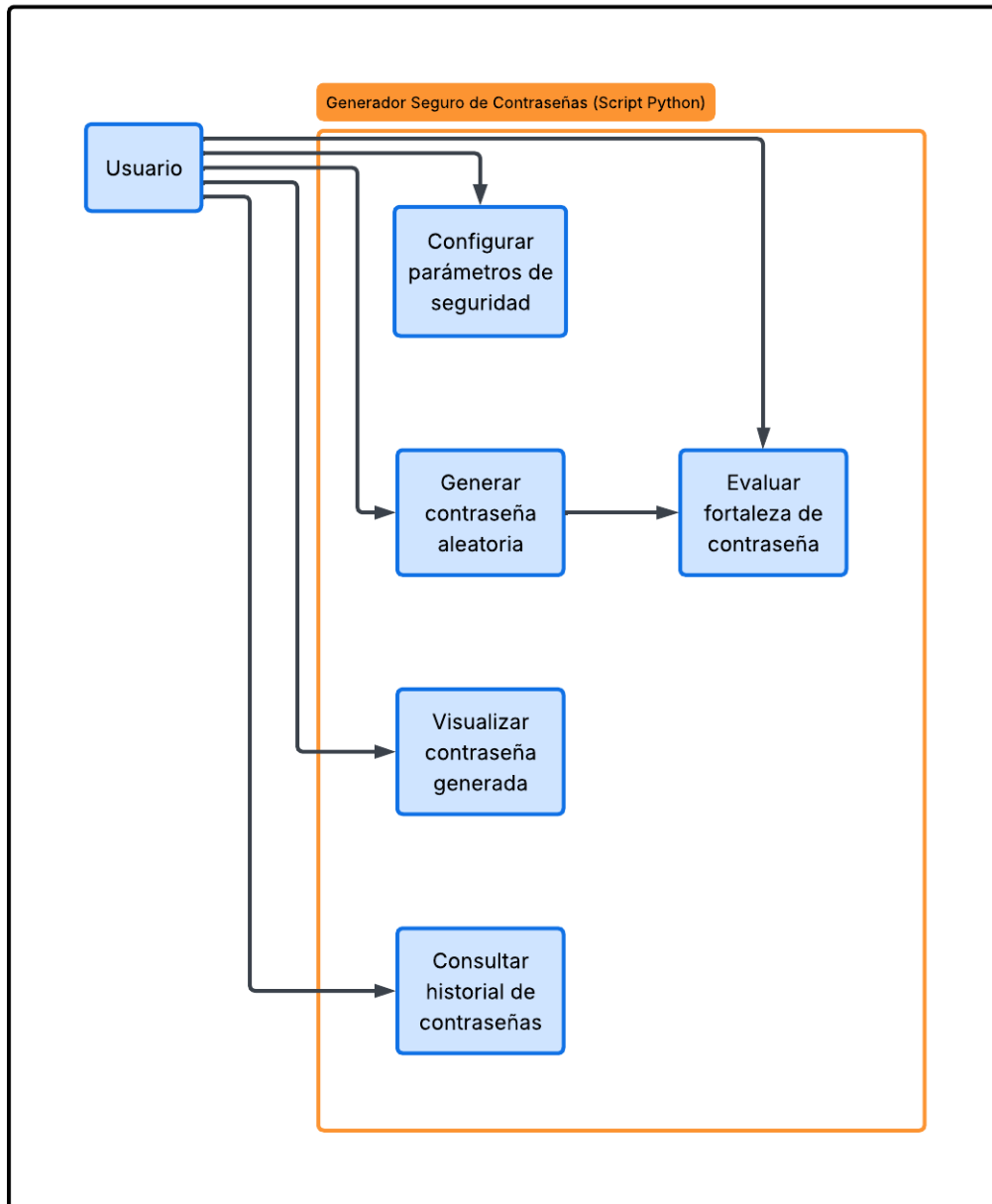
Diagrama de casos de uso

Para comprender la interacción entre el usuario y el sistema, se elaboró un diagrama de casos de uso. Este diagrama permite visualizar las funcionalidades principales desde la perspectiva del usuario final:

- Configurar contraseña: el usuario define las preferencias de generación
- Elegir longitud: selección del número de caracteres (mínimo 8, recomendado 12+)

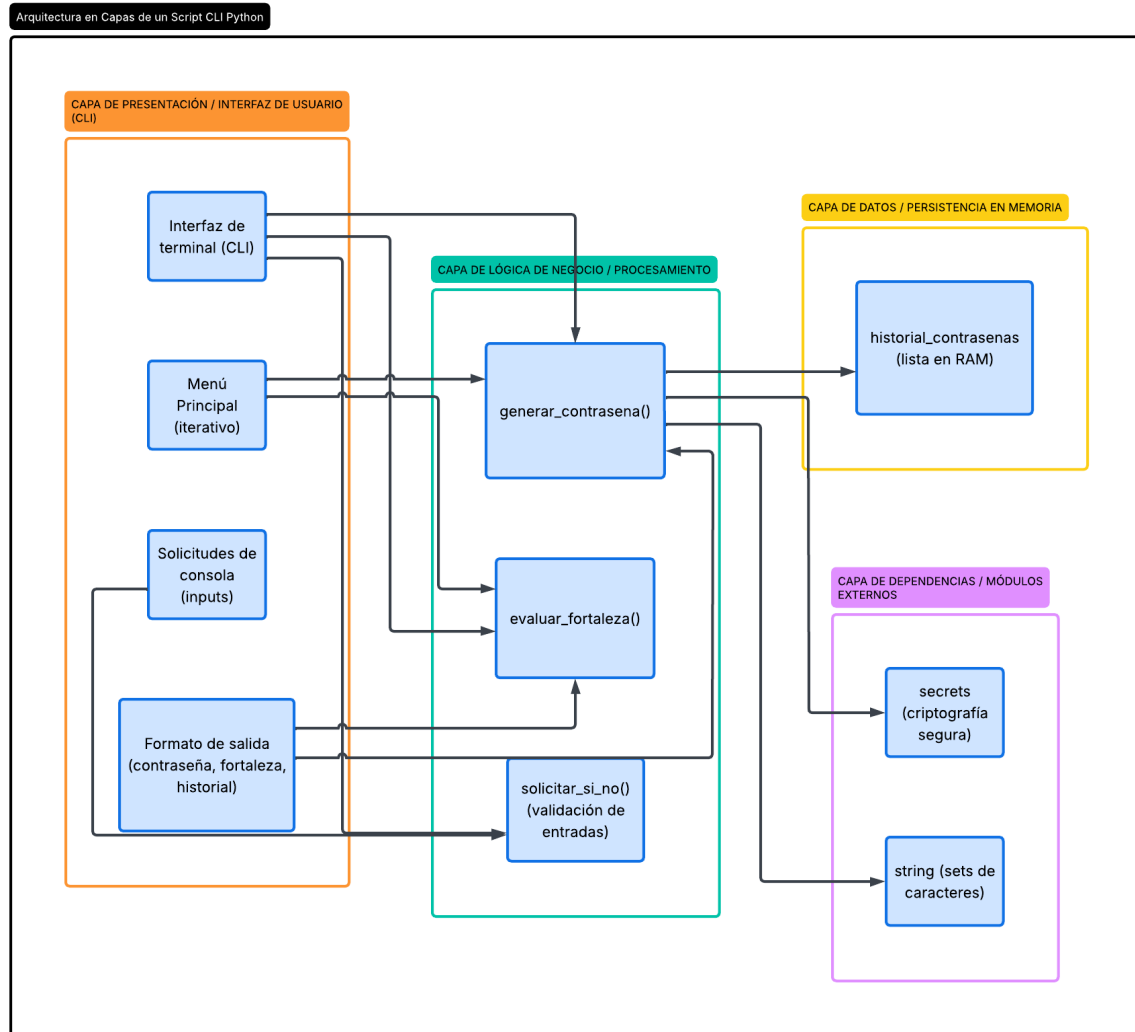
- Elegir mayúsculas, números y símbolos: personalización del conjunto de caracteres
- Generar contraseña: el sistema produce la clave aleatoria segura con secrets
- Ver historial: visualización de contraseñas generadas durante la sesión

Casos de Uso – Generador Seguro de Contraseñas



4. Diseño de Arquitectura

La arquitectura del software se basa en un modelo de 3 módulos principales que trabajan de forma secuencial y coordinada:



Interfaz de Usuario

Permite la interacción directa entre el usuario y el sistema. Desde esta capa el usuario ingresa las configuraciones deseadas, visualiza la contraseña generada y consulta el historial. Se implementa mediante la función `input ()` de Python con validación de entradas.

Lógica del Sistema

Se encarga de procesar las solicitudes del usuario. Valida las configuraciones seleccionadas, controla el flujo del programa mediante estructuras condicionales (if/elif/else) y repetitivas (while), y coordina la comunicación entre la interfaz y el módulo generador.

Generador de Contraseñas

Modulo encargado de crear las contraseñas aleatorias utilizando el módulo secrets de Python, que garantiza aleatoriedad criptográficamente segura. Combina los conjuntos de caracteres seleccionados y construye la contraseña final según los parámetros recibidos.

4.1. Relación con el Impacto Tecnológico en la Sociedad

Las nuevas tecnologías han transformado profundamente la forma en que las personas interactúan, trabajan y protegen su información en el entorno digital. En este contexto, la ciberseguridad emerge como una de las áreas más críticas y de mayor crecimiento en el campo de la ingeniería de software.

Según el Instituto Nacional de Estándares y Tecnología de Estados Unidos (NIST), el uso de contraseñas débiles o reutilizadas representa uno de los principales vectores de ataque en incidentes de seguridad informática a nivel mundial. Este proyecto responde directamente a esa problemática, demostrando como una solución de software relativamente simple puede tener un impacto real y positivo en la seguridad digital de cualquier usuario.

El Generador Seguro de Contraseñas es un ejemplo concreto de como los conocimientos adquiridos en lógica de programación pueden aplicarse para resolver problemas reales. Esta herramienta puede ser utilizada por cualquier persona, independientemente de su nivel técnico, para mejorar su seguridad digital y proteger su información personal.

A futuro, este sistema podría evolucionar hacia una aplicación con interfaz gráfica, integración con gestores de contraseñas o una extensión de navegador, ampliando aún más su alcance e impacto social.

4.2. Conclusiones

El desarrollo de este proyecto permitió aplicar de manera integral los contenidos de la asignatura de Lógica de Programación, demostrando que los fundamentos del software tienen aplicaciones directas y valiosas en problemas reales de la sociedad.

- El módulo secrets de Python garantiza aleatoriedad criptográficamente segura, validando la elección tecnológica del proyecto frente a alternativas menos seguras como random.
- Las estructuras condicionales implementadas permiten una personalización flexible de la contraseña, adaptándose a los diferentes niveles de seguridad requeridos por el usuario.
- Las estructuras repetitivas (while) con manejo de excepciones (try/except) garantizan una experiencia robusta donde el programa no falla ante entradas incorrectas.
- El sistema de evaluación de fortaleza ofrece retroalimentación educativa inmediata, fomentando buenas prácticas de seguridad digital en el usuario.
- La organización del código en funciones con responsabilidad única establece las bases del desarrollo de software profesional y escalable.

En conclusión, este proyecto demuestra que la ingeniería de software, aplicada desde sus fundamentos, tiene el potencial de generar herramientas que mejoren la seguridad y calidad de vida de las personas en el entorno digital actual.

4.3. Limitaciones e Implicaciones

4.3.1. Limitaciones del sistema

- El programa opera en entorno de consola (CLI), sin interfaz gráfica, lo que puede resultar menos intuitivo para usuarios no técnicos.

- Las contraseñas generadas no se almacenan de forma persistente entre sesiones; al cerrar el programa, el historial se pierde.
- No incluye integración directa con gestores de contraseñas externos como LastPass o Bitwarden.
- La evaluación de fortaleza es estimada cualitativamente, no calcula la entropía matemática formal de la contraseña.
- El programa no verifica si la contraseña generada ha sido comprometida previamente en bases de datos públicas.

4.3.2. Implicaciones del sistema

- Fomenta buenas prácticas de seguridad digital en usuarios de cualquier nivel técnico, contribuyendo a reducir el uso de contraseñas débiles.
- El uso de secrets asegura que las contraseñas no puedan ser predichas por atacantes, a diferencia de generadores basados en random.
- El sistema es fácilmente extensible: puede añadirse interfaz gráfica con Tkinter, version web con Flask, o integrarse como modulo en sistemas más complejos.
- El proyecto sienta las bases para el desarrollo de herramientas de ciberseguridad más avanzadas en semestres posteriores de la carrera.

5. Bibliografía

- Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.^a ed.). McGraw-Hill.
- Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson Educación.
- Larman, C. (2003). *UML y patrones: Introducción al análisis y diseño orientado a objetos* (2.^a ed.). Prentice Hall.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2006). *El lenguaje unificado de modelado UML*. Pearson Educación.
- Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley.
- Stallings, W. (2017). *Seguridad informática: Principios y práctica* (4.^a ed.). Pearson.
- NIST. (2020). *Digital Identity Guidelines*. National Institute of Standards and Technology. <https://pages.nist.gov/800-63-3/>
- Oracle. (2024). *Java Documentation*. <https://docs.oracle.com/>
- Lucid Software Inc. (2024). *Lucidchart UML Diagrams Guide*. <https://www.lucidchart.com/pages/es/que-es-un-diagrama-uml>